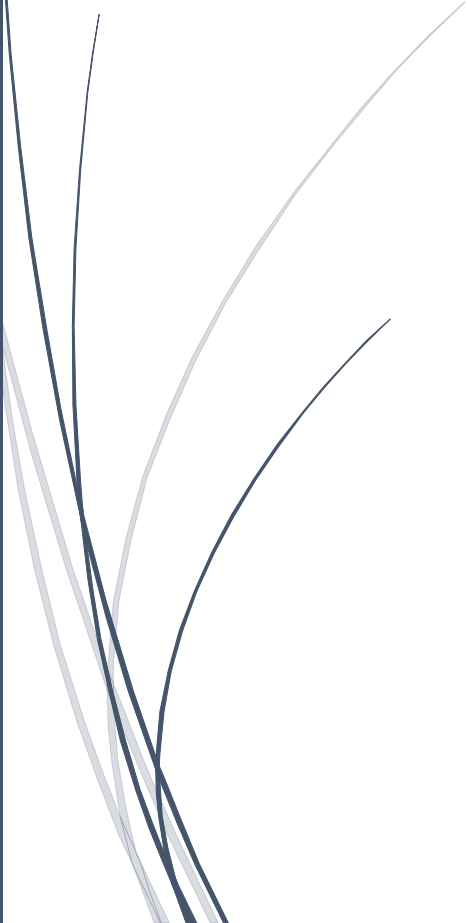


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

11/01/2026

Projet d'exploitation de données

B1-S1 Business Data Sciences

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

Mohamed Lamine Bah et Barry Thierno Ousmane
UNIVERSITE CATHOLIQUE DE L'OUEST D'ANGERS

Table des matières

I.	Créer un processus automatisé pour l'importation des données :.....	1
a)	Lecture des fichiers CSV contenant les relevés de prix.	3
b)	Suppression des prix aberrants et nuls	4
II.	Mettre à jour la table Offer :	4
a)	Créer de nouvelles offres si elles apparaissent dans les relevés et qu'elles n'existent pas encore dans la table Offer	4
b)	Mettre à jour les offres existantes.....	4
c)	Clôturer les offres qui ne sont plus actives en fonction des relevés (attention plusieurs cas possibles).....	5
III.	Ajouter un mécanisme de Log.....	7
	Création de la table Log	7
IV.	Développer un script qui vérifie l'exactitude des mises à jour apportés à la table Offer.	9

A. Introduction

Dans un contexte de forte concurrence commerciale, les chaînes de magasins doivent être en mesure d'analyser et d'exploiter rapidement les données issues de leurs relevés de prix afin d'adapter leurs offres commerciales. Ce projet s'inscrit dans cette problématique et vise à automatiser la gestion des offres à partir de relevés de prix quotidiens fournis sous forme de fichiers CSV.

L'objectif principal de ce projet est de concevoir et de mettre en œuvre un processus automatisé permettant d'importer, de contrôler et d'exploiter des données de prix provenant de plusieurs magasins du Maine-et-Loire. Ces données alimentent une table centrale nommée **Offer**, dans laquelle chaque enregistrement représente une offre commerciale active ou expirée pour un produit donné dans un magasin donné, sur une période définie.

Le projet repose sur l'utilisation du langage SQL et d'une base de données relationnelle. Il comprend plusieurs étapes clés listées dans la table de matières ci-dessous :

I. Créer un processus automatisé pour l'importation des données :

Pour automatiser l'importation de données sur SQL, nous avons besoins d'une table vide qui abritera nos données

❖ *Création de la table*

```
DROP TABLE IF EXISTS ventes1;
```

```
CREATE TABLE ventes1 (  
    EAN VARCHAR(13),  
    StoreNumberID INT,  
    RecommendedPrice DECIMAL(10,2),  
    AppliedPrice DECIMAL(10,2)  
);
```

- **AppliedPrice** : prix appliqué

Sur cette image, j'ai créé une table ventes1 tout en lui demandant de la supprimer si elle existe déjà et comme demandé dans l'énoncé les colonnes suivantes :

- **EAN** : désigne l'identifiant du produit
- **StoreNumberID** : l'identifiant du magasin
- **RecommendedPrice** : le prix recommandé

❖ Vidage de la table

```
TRUNCATE TABLE ventes1;-
```

Dans notre cas, comme l'analyse est journalière, il est recommandé d'utiliser cette fonction pour vider

la table sans la supprimer

❖ Déclaration des variables

```
DECLARE @fichier_vente NVARCHAR(50) = 'DATA20241118';  
  
DECLARE @Date_vente DATE = '2024-12-02';      -- date du fichier  
  
DECLARE @emplacement NVARCHAR(260) = 'C:\Users\HP\Documents\SQL\Projet\';  
  
DECLARE @sql NVARCHAR(MAX) =
```

Nous avons remarqué que lors de la déclaration des variables, nous avons utilisé des @, ce qui est essentielle pour automatiser et pouvoir réutiliser ces variables plus tard.

- **@fichier_vente** : nom du fichier csv ;
- **@Date_vente** : la date
- **@emplacement** : c'est l'emplacement exact du fichier

a) Lecture des fichiers CSV contenant les relevés de prix.

C'est l'étape la plus importante pour faire l'importation de données

```
DECLARE @sql NVARCHAR(MAX) =  
  
N'BULK INSERT ventes1  
  
FROM ''' + @emplacement + @fichier_vente + N'.csv'  
WITH (  
  
    FIRSTROW = 2,  
  
    FIELDTERMINATOR = ';',  
  
    ROWTERMINATOR = '0x0d0a', -- Windows (CRLF)  
  
    TABLOCK  
  
);  
  
EXEC sys.sp_executesql @sql;
```

Ici, nous avons procédé à la déclaration de la variable qui va stocker la requête SQL complète

Pour changer le fichier csv qu'on veut lire, il suffit juste de changer le nom à la déclaration dans cette requête

```
DECLARE @fichier_vente NVARCHAR(50) = 'DATA20241118';
```

```
N'BULK INSERT ventes1  
  
FROM ''' + @emplacement + @fichier_vente + N'.csv'
```

La requête "BULK INSERT" construit le chemin du fichier à importer. Dans notre cas, on a fait appel aux variables déjà déclarées qui contiennent l'emplacement(@emplacement), le nom(@fichier_vente), ensuite nous avons précisé l'extension du fichier (N'.csv')

```
WITH (  
  
    FIRSTROW = 2,  
  
    FIELDTERMINATOR = ';',  
  
    ROWTERMINATOR = '0x0d0a', -- Windows (CRLF)  
  
    TABLOCK  
  
);
```

Après avoir construit le chemin qui mène au fichier, nous avons utilisé quelques fonctions

- **FIRSTROW=2** : il commence l'importation à la deuxième ligne pour ignorer l'entête
- **FIELDTERMINATOR = ' ; '** : les colonnes sont séparées par des points virgules ;
- **ROWTERMINATOR = "0x0d0a"** : fin de ligne **Windows (CRLF)**
- **TABLOCK** : elle permet d'améliorer la performance lors du chargement

```
EXEC sys.sp_executesql @sql;
```

Cette ligne exécute la requête SQL qu'on vient de construire

b) Suppression des prix aberrants et nuls

```
delete from ventes1 where RecommendedPrice=0 or AppliedPrice=0  
or RecommendedPrice>1000 or AppliedPrice>1000
```

C'est une requête
régulièrement sur SQL

II. Mettre à jour la table Offer :

- a) Créer de nouvelles offres si elles apparaissent dans les relevés et qu'elles n'existent pas encore dans la table Offer

```
DECLARE @FileDate DATE = '2024-11-11';  
  
INSERT INTO Offer (  
    NumberID,  
    StoreNumberID,  
    StoreDescription,  
    EAN,  
    ItemDescription,  
    PeriodStart,  
    PeriodEnd,  
    Price,  
    VerifiedDate,  
    StockShortageDate  
)
```

Nous avons commencé par définir les colonnes dans lesquelles nous voulons insérer les données des offres qui seront créées avec la déclaration de la date bien sûr qui est nécessaire pour le fonctionnement de la requête

```
SELECT  
    ROW_NUMBER() OVER (ORDER BY v.EAN, v.StoreNumberID)  
    + ISNULL((SELECT MAX(NumberID) FROM dbo.Offer), 0) AS NumberID,  
    v.StoreNumberID,  
    s.description AS StoreDescription,  
    CAST(v.EAN AS bigint) AS EAN,  
    ei.ItemDescription AS ItemDescription,  
    CAST(@FileDate AS datetime) AS PeriodStart,  
    DATETIMEFROMPARTS(9999,12,31,0,0,0,0) AS PeriodEnd,  
    CAST(v.AppliedPrice AS money) AS Price,  
    CAST(@FileDate AS datetime) AS VerifiedDate,  
    NULL AS StockShortageDate  
  
FROM ventes1 v  
JOIN Store s ON s.numberid = v.StoreNumberID  
LEFT JOIN EANItem ei ON ei.ean = CAST(v.EAN AS bigint)  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM Offer o  
    WHERE o.EAN = CAST(v.EAN AS bigint)  
    AND o.StoreNumberID = v.StoreNumberID  
    AND o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0)  
)
```

Après la déclaration, nous avons insérer les données et nous remarquons certaines instructions comme :

- **OW_NUMBER ()** numérote les lignes retournées par ce SELECT (1, 2, 3...)
- **MAX(NumberID)** récupère le dernier identifiant existant
- **CAST (@FileDate AS datetime) AS PeriodStart**
DATETIMEFROMPARTS (9999,12,31,0,0,0,0) AS PeriodEnd : définit l'offre comme active à partir de @FileDate jusqu'à nouvel ordre" (9999-12-31)

b) Mettre à jour les offres existantes

```
DECLARE @Date_vente DATE = '2024-11-11';
```

```
UPDATE o
SET o.VerifiedDate = CAST(@Date_vente AS datetime)
FROM Offer o
JOIN ventes1 v
ON o.StoreNumberID = v.StoreNumberID
AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0);
```

Il met à jour la date de vérification (**VerifiedDate**) uniquement pour les offres actives (**PeriodEnd = 9999-12-31**) qui existent dans le fichier de ventes **ventes1**

c) Clôturer les offres qui ne sont plus actives en fonction des relevés (attention plusieurs cas possibles)

Il existe plusieurs possibilités pour résoudre cette partie du projet. De notre côté, nous avons procédé à l'énumération de trois étapes :

Cas 1 : Offres dont le prix a changé

- Offres actives dont le prix est différent du relevé du jour

```
-- Offres actives dont le prix est différent du relevé du jour
SELECT
    o.NumberID,
    o.StoreNumberID,
    o.EAN,
    o.Price AS OldPrice,
    v.AppliedPrice AS NewPrice,
    o.PeriodStart,
    o.PeriodEnd
FROM Offer o
JOIN ventes1 v
ON o.StoreNumberID = v.StoreNumberID
AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0)
    AND o.Price IS NOT NULL
    AND o.Price <> CAST(v.AppliedPrice AS money);

UPDATE o
SET o.PeriodEnd = CAST(@Date_vente AS datetime) -- fermeture à la date du fichier
FROM Offer o
JOIN ventes1 v ON o.StoreNumberID = v.StoreNumberID
AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0) -- uniquement offres actives
    AND o.Price IS NOT NULL
    AND o.Price <> CAST(v.AppliedPrice AS money); -- comparaison du prix
```

```

SELECT
    o.NumberID,
    o.StoreNumberID,
    o.EAN,
    o.Price AS OldPrice,
    v.AppliedPrice AS NewPrice,
    o.PeriodStart,
    o.PeriodEnd
FROM Offer o
JOIN ventes1 v
    ON o.StoreNumberID = v.StoreNumberID
AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0)
    AND o.Price IS NOT NULL
    AND o.Price <> CAST(v.AppliedPrice AS money);

```

Il compare le prix actuel de l'offre active (**o.Price**) avec le prix relevé dans les ventes du jour (**v.AppliedPrice**) pour le même magasin et le même **EAN**. Il liste uniquement les offres actives dont le prix a changé

```

UPDATE o

```

```

SET o.PeriodEnd = CAST(@Date_vente AS datetime) -- fermeture à la date du fichier
FROM Offer o
JOIN ventes1 v ON o.StoreNumberID = v.StoreNumberID
AND o.EAN = TRY_CAST(v.EAN AS bigint)

WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0) -- uniquement offres actives
    AND o.Price IS NOT NULL
    AND o.Price <> CAST(v.AppliedPrice AS money); -- comparaison du prix

-- Offres fermées aujourd'hui à cause d'un changement de prix

```

Cette requête Clôture les offres actives en mettant **PeriodEnd** à la date du fichier uniquement si le prix est différent. Les offres restent historisées et ne sont plus actives. Elles libèrent la place pour une nouvelle offre au nouveau prix

○ Offres fermées aujourd'hui à cause d'un changement de prix

```

-- Offres fermées aujourd'hui à cause d'un changement de prix

SELECT
    NumberID,
    StoreNumberID,
    EAN,
    Price,
    PeriodStart,
    PeriodEnd
FROM Offer
WHERE PeriodEnd = CAST(@Date_vente AS datetime)
ORDER BY NumberID DESC;

```

Sélectionne uniquement les offres dont la date de fin (**PeriodEnd**) = date du fichier. Ce sont donc des offres : actives hier et clôturées aujourd'hui (suite à une détection de prix différent)

Cas 2 : l'offre est absente du relevé du jour

```

DECLARE @Date_vente DATE = '2024-11-11';

UPDATE o
SET o.PeriodEnd = CAST(@Date_vente AS datetime)
FROM Offer o
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0) -- offres actives
AND NOT EXISTS (
    SELECT 1
    FROM ventes1 v
    WHERE v.StoreNumberID = o.StoreNumberID
    AND TRY_CAST(v.EAN AS bigint) = o.EAN
);

```

Il cible uniquement les offres actives `o.PeriodEnd = '9999-12-31'`. Si aucune vente n'est trouvée, alors l'offre est clôturée à la date du fichier (`o.PeriodEnd = @Date_vente`).

Cas : Le magasin est fermé

---Cas 3: Le magasin est fermé

```

UPDATE o
SET o.PeriodEnd = CAST(@FileDate AS datetime)
FROM Offer o
JOIN Store s ON s.numberid = o.StoreNumberID
WHERE o.PeriodEnd = CAST('9999-12-31T00:00:00' AS datetime)
AND s.closingdate IS NOT NULL
AND s.closingdate <= @FileDate;

SET @ClosedStoreClosed = @@ROWCOUNT;

```

- Mettre à jour toutes les offres dans la table **Offer**.
- Joindre chaque offre avec sa boutique correspondante dans la table **Store** (**StoreNumberID**).

- Filtrer uniquement les offres actives (**PeriodEnd = 9999-12-31**).
- Vérifier que la boutique a une date de fermeture (**closingdate IS NOT NULL**) et que cette date est antérieure ou égale à la date du fichier (**@FileDate**).
- Si ces conditions sont remplies, mettre à jour **PeriodEnd** de l'offre avec la date du fichier, ce qui ferme l'offre.
- Enregistrer dans la variable **@ClosedStoreClosed** le nombre de lignes affectées par cette mise à jour (**@@ROWCOUNT**).

III. Ajouter un mécanisme de Log

L'objectif de ce log est de faciliter l'audit et le contrôle qualité, de diagnostiquer rapidement toute anomalie et de garantir que les règles métier sont respectées. Ce mécanisme constitue ainsi un outil essentiel pour la transparence, la maintenance et le suivi du cycle de vie des offres.

Création de la table Log

```

---Création de la table des log---

IF OBJECT_ID('ProcessLog', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.ProcessLog (
        LogID INT IDENTITY(1,1) PRIMARY KEY,          -- Identifiant auto
        LogDate DATETIME NOT NULL DEFAULT GETDATE(),   -- Date/heure exécution

        nom_fichier NVARCHAR(50) NOT NULL,             -- Nom fichier (ex: DATA20241125)
        date_fichier DATE NOT NULL,                    -- Date fichier

        ligne_importé INT NOT NULL,                    -- Lignes importées
        ligne_rejeté INT NOT NULL,                     -- Lignes rejetées au nettoyage

        nouvelle_offre INT NOT NULL,                   -- Offres créées
        offre_m_a_j INT NOT NULL,                      -- Offres mises à jour (VerifiedDate)

        offre_fer_c INT NOT NULL,                       -- Offres fermées (prix changé)
        offre_fer_a INT NOT NULL,                       -- Offres fermées (absentes)
        offre_fer_mf INT NOT NULL                      -- Offres fermées (magasin fermé)
    )
END;

```

Nous constatons que la table a été créée avec les colonnes observées sur l'image

```

---Ajout d'un statut---

IF COL_LENGTH('ProcessLog', 'Status') IS NULL
BEGIN
    ALTER TABLE ProcessLog
    ADD Status NVARCHAR(20) NOT NULL
    CONSTRAINT DF_ProcessLog_Statut DEFAULT 'Succès';
END;

```

Ce code vérifie si la colonne **Status** existe déjà dans la table **ProcessLog**. **COL_LENGTH('ProcessLog', 'Status') IS NULL** renvoie vrai si la colonne n'existe

pas. Si la colonne n'existe pas, modifie la table **ProcessLog** pour ajouter la colonne **Status** de type **NVARCHAR(20)** et obligatoire (**NOT NULL**). Crée une contrainte par défaut (**DEFAULT 'Succès'**) pour la colonne **Status** afin que toutes les nouvelles lignes qui ne fournissent pas de valeur aient automatiquement la valeur 'Succès'.

```

---Ajout d'un ErrorMessage ---

IF COL_LENGTH('ProcessLog', 'ErrorMessage') IS NULL
BEGIN
    ALTER TABLE ProcessLog
    ADD ErrorMessage NVARCHAR(4000) NULL;
END;

```

Ce code vérifie si la colonne **ErrorMessage** existe déjà dans la table **ProcessLog**. **COL_LENGTH('ProcessLog', 'ErrorMessage') IS**

NULL renvoie vrai si la colonne n'existe pas. Si la colonne n'existe pas, modifie la table **ProcessLog** pour ajouter la colonne **ErrorMessage** de type **NVARCHAR (4000)** et optionnelle (**NULL**).

```

---Index : 1 seul Succès par (nom_fichier, date_fichier)---
--Pour créer l'index unique,il fallait identifier les doublons et les supprimer

--Identification des doublons---

SELECT StoreNumberID, EAN, COUNT(*) AS Nb_Doublons FROM Offer
WHERE PeriodEnd = '9999-12-31T00:00:00'
GROUP BY StoreNumberID, EAN
HAVING COUNT(*) > 1;

--Suppression des doublons---

DELETE FROM Offer WHERE NumberID IN (
    SELECT NumberID
    FROM (
        SELECT NumberID,
            ROW_NUMBER() OVER (PARTITION BY StoreNumberID, EAN, PeriodEnd ORDER BY NumberID) AS rn
        FROM Offer
        WHERE PeriodEnd = '9999-12-31T00:00:00'
    ) x
    WHERE rn > 1
);

```

Nous remarquons ici la création d'un index unique qui nécessitait l'identifications et la suppression de doublons. C'est ce qui nous montre cette image

```

--création de l'index unique---

IF NOT EXISTS (
SELECT 1
FROM sys.indexes
WHERE name = 'Unique_Offer_Active'
      AND object_id = OBJECT_ID('Offer')
)
BEGIN
CREATE UNIQUE INDEX Unique_Offer_Active ON Offer (StoreNumberID, EAN)
WHERE PeriodEnd = '9999-12-31T00:00:00';
END;

```

- L'index est **unique**, ce qui empêche les doublons
- L'unicité s'applique sur le couple (**StoreNumberID, EAN**)
- L'index est **filtré** : il ne concerne que les lignes où **PeriodEnd = '9999-12-31T00:00:00'**

Cette requête vérifie

- Si l'index **Unique_Offer_Active** existe déjà sur la table **Offer**
- Si l'index **n'existe pas**, elle le crée

IV. Développer un script qui vérifie l'exactitude des mises à jour apportés à la table Offer.

C'est la dernière étape du projet qui consiste à vérifier que l'ensemble des modifications, c'est-à-dire mise à jour qui ont été effectuées sur la table Offer ont réussi. Pour cela, observons ces scripts :

```

-- A) Doublons d'offres actives--
DECLARE @Date_vente DATE = '2024-11-11';
SELECT StoreNumberID, EAN, COUNT(*) AS NbOffresActives
FROM Offer
WHERE PeriodEnd = '9999-12-31T00:00:00'
GROUP BY StoreNumberID, EAN
HAVING COUNT(*) > 1;

```

date de fin d'offre "active" (**PeriodEnd = '9999-12-31'**). Il renverra aucune ligne, le nettoyage a été déjà fait

Ce script a pour but de détecter les doublons dans la table Offer pour les offres actives. Un doublon se définit ici comme une combinaison magasin (**StoreNumberID**) + produit (**EAN**) qui apparaît plusieurs fois avec une

```

---B) Prix aberrants en actif (attendu : 0 ligne)---
SELECT StoreNumberID, EAN, Price, PeriodStart, PeriodEnd, VerifiedDate
FROM dbo.Offer
WHERE PeriodEnd = '9999-12-31T00:00:00'
AND (Price IS NULL OR Price <= 0 OR Price > 1000);

```

toutes les offres actives ont un prix valide et réaliste.

Ce script permet de détecter les offres actives dont le prix est incorrect ou incohérent. L'objectif est de s'assurer que

---C) Magasin fermé mais offre active (attendu : 0 ligne)---

```
DECLARE @Date_vente DATE = '2024-11-11';
SELECT o.StoreNumberID, o.EAN, s.closingdate, o.PeriodEnd
FROM dbo.Offer o
JOIN dbo.Store s
    ON s.numberid = o.StoreNumberID
WHERE o.PeriodEnd = '9999-12-31T00:00:00'
    AND s.closingdate IS NOT NULL
    AND s.closingdate <= @Date_vente;
```

Ce script permet de repérer les offres encore actives (**PeriodEnd** = '9999-12-31') pour lesquelles le magasin associé est fermé (**closingdate** non nul et antérieur ou égal à la date du fichier **@Date_vente**).

----D) Offre dont le prix est différent de celui du relevé du jour(0 offre attendue)

```
DECLARE @Date_vente DATE = '2024-11-11';
SELECT
    COUNT(*) AS Nb_Offres_Actives_EcartPrix
FROM Offer o
JOIN ventes1 v
    ON o.StoreNumberID = v.StoreNumberID
    AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0)
    AND o.Price <> CAST(v.AppliedPrice AS money);

SELECT
    COUNT(*) AS Nb_Offres_Fermes_EcartPrix
FROM Offer o
JOIN ventes1 v
    ON o.StoreNumberID = v.StoreNumberID
    AND o.EAN = TRY_CAST(v.EAN AS bigint)
WHERE CAST(o.PeriodEnd AS date) = @Date_vente
    AND o.Price <> CAST(v.AppliedPrice AS money);
```

- La première requête compte le nombre d'offres actives dont le prix diffère du relevé du jour. Elle sélectionne les offres encore actives (**PeriodEnd** = 9999-12-31) et compare le prix stocké (**Price**) avec le prix relevé dans ventes1 (**AppliedPrice**).
- La seconde requête compte le nombre d'offres déjà fermées à cause d'un écart de prix. Elle

considère les offres dont la date de fin (**PeriodEnd**) correspond à la date du fichier (**@Date_vente**) et dont le prix diffère toujours du relevé.

---F) l'offre est absente du relevé du jour

```
DECLARE @Date_vente DATE = '2024-11-11';
SELECT
    COUNT(*) AS Nb_Actives_AbsentesReleve
FROM Offer o
WHERE o.PeriodEnd = DATETIMEFROMPARTS(9999,12,31,0,0,0,0)
    AND NOT EXISTS (
        SELECT 1
        FROM ventes1 v
        WHERE v.StoreNumberID = o.StoreNumberID
            AND TRY_CAST(v.EAN AS bigint) = o.EAN
    );
```

- Il sélectionne toutes les offres dont la date de fin (**PeriodEnd**) correspond à une offre active (9999-12-31) et qui n'ont pas de correspondance dans la table ventes1 pour le même magasin (**StoreNumberID**) et produit (**EAN**).

- La clause **NOT EXISTS** sert à identifier les offres absentes du relevé.

Objectif : s'assurer que toutes les offres qui ne figurent pas dans le fichier ont été identifiées pour être éventuellement fermées. Le résultat attendu est 0 ligne si toutes les offres ont été correctement mises à jour.

B. Conclusion

Ce projet a permis de développer un processus automatisé complet pour l'importation et la mise à jour des données de ventes dans la table Offer. Chaque étape a été soigneusement conçue pour garantir l'intégrité des informations : création de nouvelles offres, mise à jour des offres existantes, fermeture des offres obsolètes ou incorrectes, et traitement des magasins fermés.

L'implémentation d'un mécanisme de log a permis de suivre précisément chaque opération, de détecter les erreurs et de fournir un historique fiable des traitements. Les scripts de vérification ont assuré la cohérence des données, en contrôlant les doublons, les prix aberrants, les écarts avec les relevés journaliers et la fermeture appropriée des offres.

En conclusion, le projet garantit que les offres actives reflètent exactement la réalité des ventes, tout en offrant une traçabilité complète des mises à jour et des anomalies. Ce dispositif fournit une base solide pour la prise de décision, la fiabilité des rapports et l'optimisation future des processus commerciaux.

C. Bibliographie

Dans le cadre de la réalisation de ce projet, nous avons fait appel aux différentes notions vues durant le semestre. Nous avons aussi fait appel à l'intelligence artificielle sur certaines étapes, surtout au niveau du Log que nous ne pensons pas même fini de réaliser.

Pour finir, ça été un plaisir pour nous de réaliser ce projet et nous espérons qu'il sera à la hauteur de vos attentes.